

Exame de Programação Funcional

27 de Junho de 2010

DCC/FCUP

2ª Parte

Nome: _____

Nº mecanográfico: _____

- Esta 2ª parte contém 5 questões e 3 páginas.
- Responda às questões 1 e 2 no espaço marcado no enunciado.
- Responda às questões 3, 4 e 5 numa folha de exame separada.

1. (15%) Escreva uma definição da função *crescente* :: $[Int] \rightarrow Bool$ que verifica se uma lista de inteiros está por ordem crescente. Exemplos: *crescente* [2, 5, 3] = *False*; *crescente* [1, 3, 3, 5] = *True*. Deve considerar que a lista vazia e as listas com um só elemento estão por ordem crescente.

2. (15%) Considere um polinómio $P(X) = c_0 + c_1X + \dots + c_nX^n$ representado pela lista dos seus coeficientes $[c_0, c_1, \dots, c_n]$. Podemos calcular o valor do polinómio num ponto de forma eficiente usando a *forma de Horner*:

$$\begin{aligned} P(z) &= c_0 + c_1z + \dots + c_nz^n \\ &= c_0 + z \times (c_1 + z \times (\dots + z \times (c_{n-1} + z \times c_n) \dots)) \end{aligned} \quad (1)$$

Note que usando a expressão (1) não necessitamos de calcular potências: para calcular o valor dum polinómio de grau n usamos apenas n adições e n produtos.

- (a) Complete a seguinte definição recursiva tal que *horner cs z* calcula o valor do polinómio com lista de coeficientes *cs* no ponto *z* usando a forma de Horner.

```
horner :: [Double] -> Double -> Double

horner [] z = 0

horner (c:cs) z = _____
                  _____
```

- (b) A forma de Horner também pode ser expressa como aplicação da função de ordem superior *foldr*. Complete a definição seguinte de forma a que a igualdade seja correcta.

```
horner cs z = foldr _____ cs
```

Responda às questões 3, 4 e 5 numa folha de exame separada.

- 3.** (20%) Considere um tipo de dados em Haskell para representar proposições lógicas.

```
data Prop = Var Char
          | Neg Prop
          | Conj Prop Prop
          | Disj Prop Prop
```

Escreva uma função `contar :: Prop → [(Char, Int)]` cujo resultado é uma lista de associações entre cada variável e o número de vezes que ocorre na proposição. Exemplo: `contar (Conj (Var 'a') (Disj (Var 'b') (Var 'a')))` = `[('a', 2), ('b', 1)]` (a ordem dos pares no resultado não é importante).

Pode e deve resolver o problema usando funções auxiliares; tenha o cuidado de definir todas as funções que não façam parte do prelúdio-padrão.

- 4.** (15%) Usando indução sobre listas, mostre que

$$\text{map } f \text{ } (xs \text{++} ys) = \text{map } f \text{ } xs \text{++} \text{map } f \text{ } ys$$

para quaisquer listas finitas xs, ys . Sugestão: comece por escolher uma variável de indução.

- 5.** (15%) Considere a definição em Haskell dum tipo de dados para árvores binárias com valores nos nós:

```
data Arv a = Vazia | No a (Arv a) (Arv a)
```

- (a) Escreva uma definição recursiva da função `listar :: Arv a → [a]` que liste os valores dos nós de uma árvore por ordem infixada da esquerda para a direita.
- (b) Escreva uma definição recursiva da função `mapArv :: (a → b) → Arv a → Arv b` que aplique uma função ao valor de cada nó de uma árvore binária.
- (c) Usando indução sobre árvores, mostre que

$$\text{listar } (\text{mapArv } f \text{ } t) = \text{map } f \text{ } (\text{listar } t)$$

para qualquer função total f e árvore finita t . Sugestão: pode usar a propriedade expressa no Exercício 4 (mesmo que não a tenha provado).

Anexo: funções do prelúdio-padrão

Além das funções sobre tipos básicos (e.g. operações aritmética), pode usar sem definir as seguintes funções sobre listas do prelúdio padrão.

<i>++</i>	<i>!!</i>	<i>head</i>	<i>tail</i>	<i>last</i>	<i>init</i>
<i>take</i>	<i>drop</i>	<i>takeWhile</i>	<i>dropWhile</i>	<i>elem</i>	<i>reverse</i>
<i>length</i>	<i>concat</i>	<i>replicate</i>	<i>repeat</i>	<i>cycle</i>	<i>iterate</i>
<i>sum</i>	<i>product</i>	<i>and</i>	<i>or</i>	<i>all</i>	<i>any</i>
<i>zip</i>	<i>zipWith</i>	<i>map</i>	<i>filter</i>	<i>foldr</i>	<i>foldl</i>